

OLLY

CNCF 오픈소스 기반 LLM 운영 관측성 MVP

비용 · 지연 · 병목 · 실패를 request_id 와 trace_id 로 추적하는 관측성 플랫폼

● Kubernetes

● OpenTelemetry

● Prometheus

● Jaeger

오픈소스SW분석 (클라우드) · 5조

최호준 32215116 · 이용찬 32213336 · 박주희 32221902 · 조하은 32234364

NORMAL p50

1.47s



HIGH_TOKEN p50

34.58s



ERROR 추적

100%



RECENT TRACES

req_6cc69475

error

req_b7a3a1ba

success

우리가 풀려던 문제: LLM은 일반 API와 다르다

사용자는 "느리다.실패했다"만 보지만, 운영자는 검색·모델 호출·토큰 증가 중 어느 단계인지 구분 필요



기존 APM 모니터링

요청이 성공했는지, 얼마나 걸렸는지 중심



HTTP Status

성공 / 실패 여부



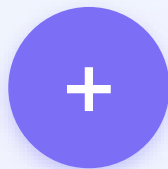
Latency

전체 응답 시간



Error rate

오류 비율



LLM 운영에 필요한 관측

성공해도 비용·토큰·병목 단계까지 알아야 함



Latency



Token



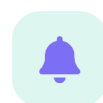
Cost



Stage duration



Error



Alert

HTTP 200이어도 운영 관점에서는 성공이 아님. OLLY의 목표는 단순 모니터링이 아니라, 요청 하나를 따라가며 원인을 좁히는 것

1차 발표 이후 바뀐 점



01

아이디어 → 실행 환경

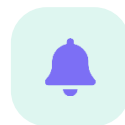
Ollama gemma3:1b 기반 샘플 /chat API를 만들고 반복 검증이 가능하게 했습니다.



02

기능 설명 → 원인 추적

request_id와 trace_id를 응답, metric, trace에 함께 남겼습니다.



03

화면 안 알림 → 운영 알림

Prometheus alert와 Discord webhook으로 이상 신호를 외부 채널까지 보냈습니다.



발표의 초점도 바뀌었습니다. “무엇을 만들겠다”가 아니라 “문제를 주입했고, 원인을 끝까지 추적했다”를 보여줍니다.

팀의 판단: 범위를 줄이고 관측성에 집중

AI · 초기안

01

- 외부 LLM API 중심 구조
- 로그인 / 권한 / 관리자 기능 포함
→ 기능 범위가 넓어져 관측성 핵심이 흐려질 위험



팀의 판단

02

- 로컬 SLM 선택:
시연 안정성 · 비용 · 반복 검증 우선
- 일반 SaaS 기능 제외:
운영 관측성과 직접 관련 낮음
- 요청 단위 관측성 집중:
request_id / trace_id 중심으로 범위 고정



완수 결과

03

- gemma3:1b 기반 재현 가능한 데모
- request_id / trace_id 기반 요청 추적
- latency / token / cost / error metric 수집
- Dashboard → Trace → Metric → Alert 흐름 완성

OLLY로 얻는 운영 효과

LLM 요청을 운영자가 추적 가능한 데이터로



요청 단위 추적

01

request_id와 trace_id로 하나의 요청을
Dashboard · Jaeger · Prometheus까지 연결



병목 원인 분리

02

전체 latency만 보는 것이 아니라
retrieve · llm_call · postprocess 중 느린 단계를 확인



비용·토큰 가시화

03

input/output token과 cost를 기록해
토큰 과다 사용과 비용 증가 원인을 파악



이상 상황 대응

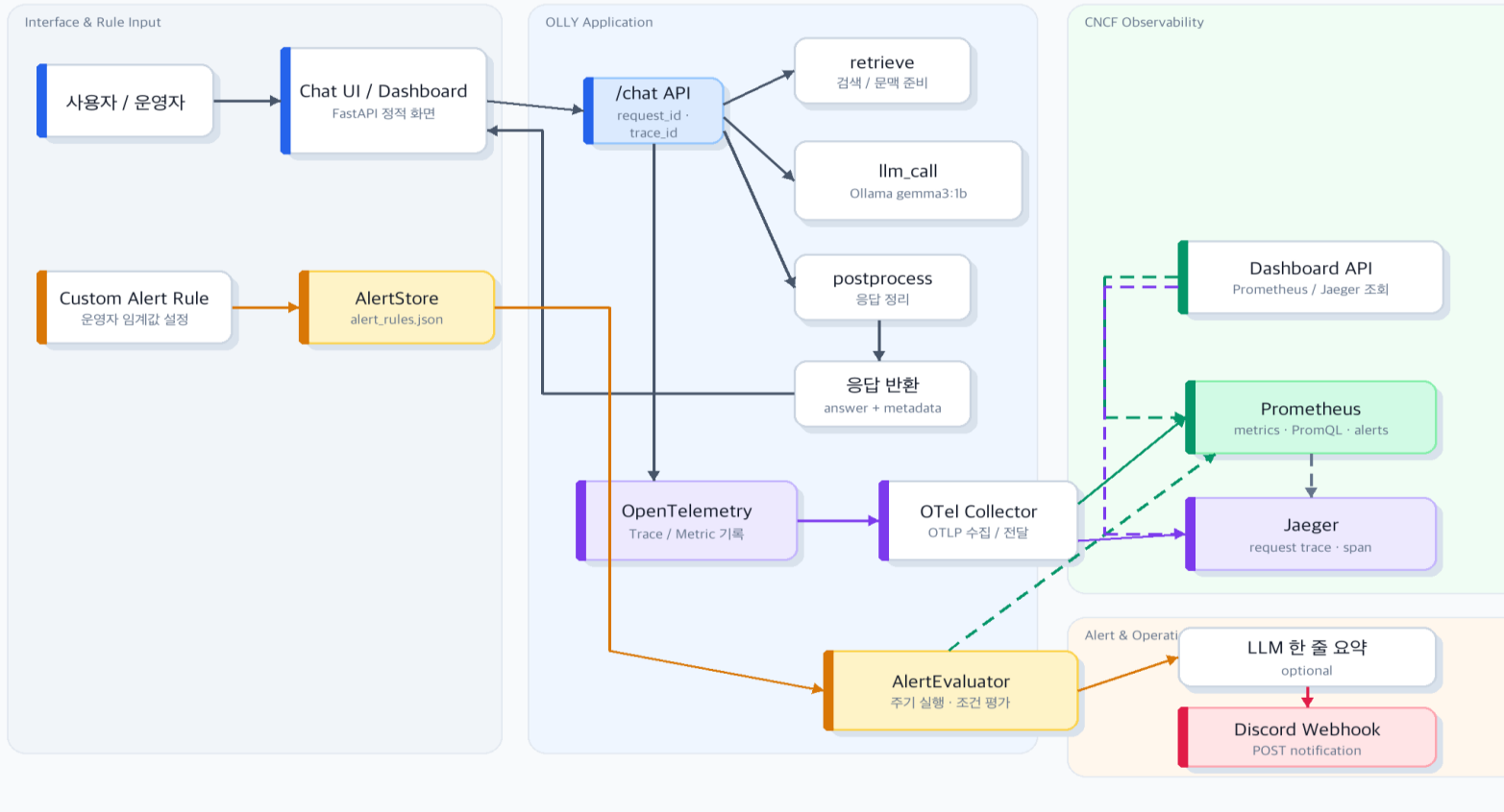
04

error나 threshold 초과 상황을 감지하고
Discord 알림으로 운영자에게 전달

최종 아키텍처

OLLY 전체 아키텍처

사용자 요청 처리, 관측성 수집, 운영 분석과 알림 흐름



요청 처리

Chat UI에서 /chat API로 들어온 요청이 retrieve, llm_call, postprocess를 거쳐 span으로 분리

관측 수집

OpenTelemetry와 Prometheus가 trace와 metric을 따로 수집

운영 분석

대시보드와 분석 챗봇은 저장된 데이터를 조회해 원인을 설명

핵심은 서비스 로직과 관측 파이프라인을 분리한 것

CNCF 프로젝트가 실제로 맡은 역할

도구 이름을 나열한 것이 아니라, 각 도구가 관측성 파이프라인의 한 역할을 맡도록 설계



Kubernetes

전체 스택 배포와 서비스 연결

컨테이너 기반 재현 가능한 실행
환경



OpenTelemetry

/chat 요청과 내부 stage 계측

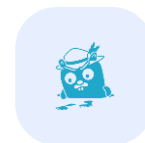
trace_id 기반 요청 흐름 생성



Prometheus

latency·token·cost·error metric
저장

운영 지표 조회·alert rule 기반



Jaeger

요청 단위 trace 시각화

retrieve·llm_call·postprocess
병목 확인

NOT CNCF



Grafana

Prometheus metric 시각화

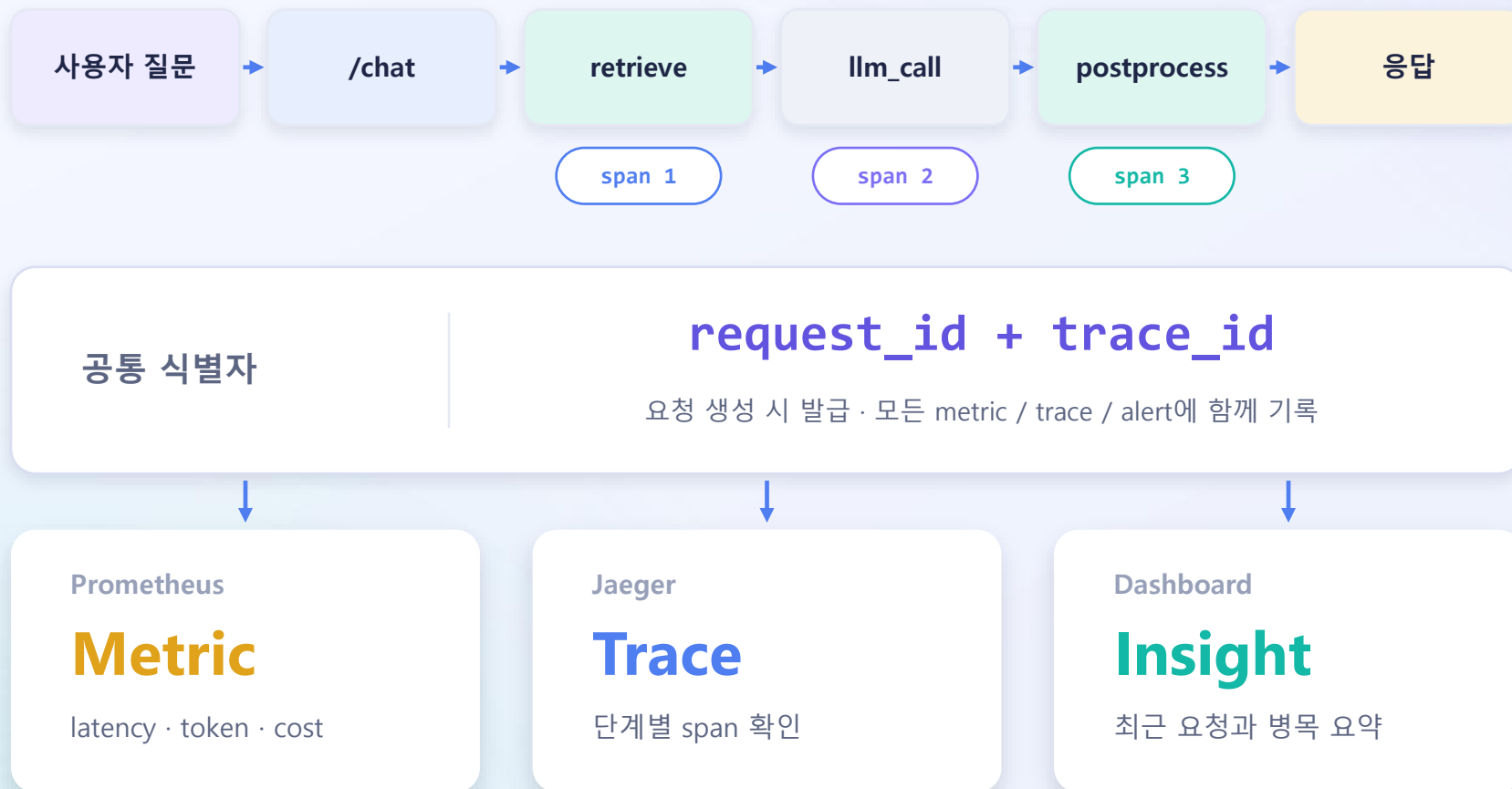
외부/생태계 도구

latency·token·cost 변화 확인



Discord Webhook 은 알림 흐름의 외부 알림 채널 로 사용

요청 하나를 끝까지 연결하는 방식



OLLY 연결 로직

- 1 ID 생성**
request_id · trace_id
- 2 Span 분리**
retrieve · llm_call · postprocess
- 3 Metric 기록**
latency · token · cost · error
- 4 동일 ID 연결**
Dashboard · Jaeger · Prometheus
- 5 Alert 전송**
threshold → Discord

데모를 위해 정의한 4가지 운영 상황

병목 · 비용 증가 · 실패를 재현하고, 관측 데이터로 확인

slow_llm



LLM 응답 지연

관측 결과

llm_call span 증가

병목 단계 확인

slow_retrieve



검색 단계 병목

관측 결과

retrieve span 증가

병목 단계 확인

high_token



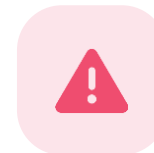
토큰 과다 사용

관측 결과

token-cost 증가

비용 지표로 확인

error



실패 요청 추적

관측 결과

error metric·trace

alert 연결 확인



Demo: 4가지 운영 상황을 관측 흐름으로 연결

localhost:8001/chat-ui

Oilly
LLM 서비스 상태를 질문으로 확인

시나리오 선택

- 일반 요청
기본 상태를 확인할 때
- RAG 검색 지연
문서 검색이 느린 상황
- 모델 응답 지연
LLM 응답이 느린 상황
- 토큰 과다 사용
비용이 늘어나는 상황
- 실패 요청
에러 추적을 보여줄 때

빠른 질문

- 현재 상태 요약해줘
- 어제랑 비교해서 비용이 왜 늘었어?
- 토큰 제일 많이 쓰는 기능 뭐야?
- RAG가 느린 거야, 모델이 느린 거야?
- 가장 느린 요청 목록 보여줘
- 최근 알림 떠 있어?

1. Chat UI 화면

Oilly 분석 도우미

첫 번째 트레이스를 기다리는 중이에요.
질문을 보내면 Oilly가 응답 시간, 토큰, 비용, 트레이스 ID를 함께 정리해드릴게요.

상태 정상 요청 ID - 트레이스 ID - 응답 시간 6ms 토큰 0 비용 \$0.000000 모델 gemma3:1b

대시보드에서 보기

예: 비용이 왜 늘었어? / RAG가 느린 거야, 모델이 느린 거야?

데모에서 운영자가 얻는 판단

원인을 단계로 좁힐 수 있다는 점이 핵심

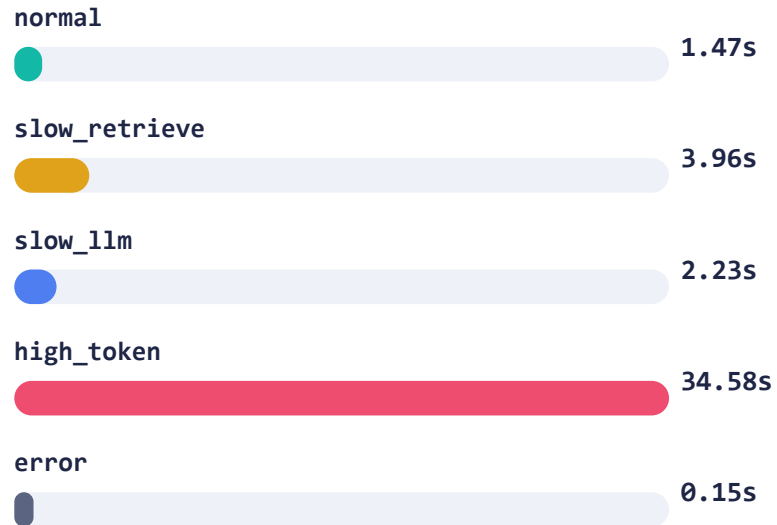
시나리오	관측 결과	운영자가 할 수 있는 판단
slow_llm	llm_call span 증가	 모델 리소스·외부 LLM API 지연 점검
slow_retrieve	retrieve span 증가	 RAG·문서 검색 계층 점검
high_token	token·cost 증가	 프롬프트 템플릿·검색 문맥 과다 주입 점검
error	error metric·trace 생성	 실패 요청의 원인 trace 추적

시나리오별 관측 신호 검증

반복 부하를 통해 latency · span · token 지표의 분리 관측을 확인 (각 시나리오 10회 반복 실행 · 총 50개 요청 기준)

시나리오별 p50 Latency 비교

단위: 초



단계·토큰 진단

RAG 검색 지연

retrieve span · 정상 대비 약 12배

1.80s

모델 호출 지연

llm_call 구간이 전체 지연의 대부분

1.99s

토큰 과다

평균 토큰 수 · high_token p50 34.58s

457

측정의 목적은 성능 자량이 아니다. Latency 증가가 retrieve 병목인지, llm_call 지연인지, 토큰 과다인지 분리해 보여주는 것이 핵심

End-to-End 검증

요청 생성부터 metric, trace, alert까지 이어지는 End-to-End 관측 흐름을 검증



ID 유지

Chat UI에서 생성된 request_id·trace_id가 Dashboard와 Jaeger에서 동일하게 확인됨



시나리오 일치

Slow llm은 llm_call span 증가
Slow_retrieve는 retrieve span 증가
High_token은 token·cost 증가



실패 보존

실패 요청도 사라지지 않고 status=error로 남고
Discord alert와 trace 추적으로 연결됨

시나리오	기대 신호	확인 결과
normal	id·latency·token·cost metadata 생성	 Chat UI와 Dashboard에 동일 metadata 표시
slow_retrieve	retrieve stage duration 증가	 Trace Detail / Jaeger에서 retrieve span 증가 확인
high_token	token·cost 증가	 Dashboard / Grafana에서 증가 확인 · 평균 457토큰
error	error metric·trace·alert 생성	 Status=error 유지 · Discord 알림 수신 · trace 추적 가능

한계 및 향후 과제



이번 MVP의 범위

- 요청 단위 추적 (request_id / trace_id)
- metric · trace 수집
- stage별 병목 확인
- Discord 외부 알림
- 운영 관측성 흐름 검증에 집중



데이터 · 연동

- 실제 Vector DB · RAG 서비스 연동
- 서비스별 stage mapping 설정화



저장 · 운영

- 장기 저장소 · retention 정책
- 장기 metric 저장 · SLO 기준 정의



보안

- raw prompt 대신 템플릿 ID·토큰 수·익명화 식별자 우선
- Secret · RBAC · TLS · Ingress · 감사 로그



품질 평가 (향후)

- RAG 근거 일치도 · 인용 검증
- LLM-as-a-judge · semantic quality monitoring



현재는 로컬 SLM 기반 MVP라 외부 API 과금·장기 저장은 검증하지 않음. 다만 request_id · trace_id 기반 구조는 그대로 확장 가능

결론

LLM 시대의 운영 핵심은 “요청 단위 운영 가시성”

OLLY는 LLM 요청 하나를 비용 · 지연 · 병목 · 실패 · 알림까지 연결해 보여주는 관측성 MVP

핵심 성과



1

요청 연결

request_id / trace_id로
Chat UI · Dashboard · Jaeger · Prometheus
연결



2

CNCF 접목

Kubernetes · OpenTelemetry · Prometheus ·
Jaeger를
LLM 운영 흐름에 적용



3

운영 판단

느린 요청, 비용 증가, 실패 요청을
데이터 기반으로 추적

즉, OLLY는 요청을 request_id와 trace_id로 끝까지 추적할 수 있게 하는 **LLM 서비스 운영을 위한 관측성 계층**입니다.

Appendix: error 시나리오 End-to-End 검증 근거 화면

[1. Chat UI]

Error 요청 발생, request_id / trace_id 생성

olly가 뭐야?

🚨 Ollly가 문제를 감지했어요.

이 요청은 실패 요청 시나리오를 보여주기 위해 일부러 실패로 기록했습니다.

- 요청 ID: req_951a7735
- 트레이스 ID: af989dfe6b2af26814a5f6e2ade12e2d
- status: error

즉, 지금 화면은 모델이 답을 몰라서 실패한 것이 아니라 OLLY가 실패 요청도 관측할 수 있는지 보여주는 데모입니다.
운영자 대시보드의 최근 요청과 Jaeger 트레이스에서 이 요청을 확인하면 실패 요청도 추적 대상에 남는 것을 볼 수 있습니다.

상태 실패

요청 ID req_951a7735

트레이스 ID af989dfe6b2af26814a5f6e2ade12e2d

응답 시간 175ms

토큰 112

비용 \$0.000000

모델 gemma3:1b

대시보드에서 보기

동일 request_id와 trace_id가 Chat UI · Dashboard · Jaeger까지 유지되고, error alert가 Discord로 전달되는지 확인

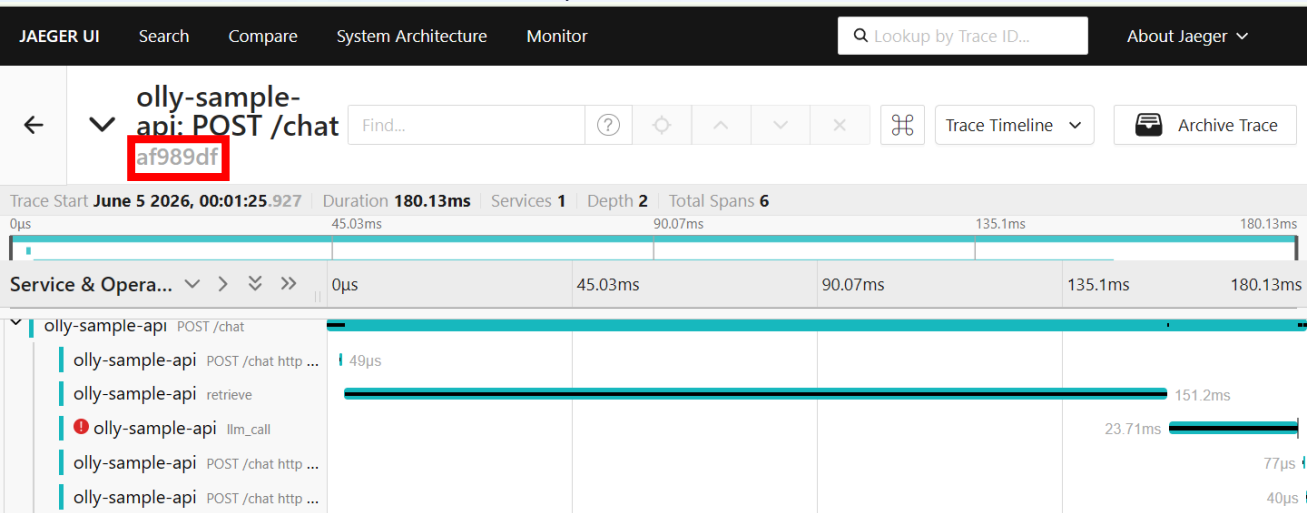
[2. Dashboard]

같은 request_id가 ERROR 상태로 보존

요청 목록 1건						
request id, trace id 검색						
시간	요청 ID	트레이스 ID	기능	시나리오	응답 시간	상태
2분 전	req_951a7735	af989dfe6b2af26814a5f6e2ade12e2d	챗봇	실패 요청	180ms	실패

[3. Jaeger]

같은 trace_id의 POST /chat trace와 error span 확인



[4. Discord]

Error rate alert가 Discord 외부 알림으로 전달

2026년 6월 4일

신규

Olly 앱 오후 3:58

[OLLY] 실패 요청 감지

지표: 에러율 (percent)

현재값: 3.3440

임계값: 0.0 초과

평가 윈도우: 1m

AI 요약: 이 요청이 실패한 이유는 데모에서 error 시나리오로 강제로 실패를 발생시키기 때문입니다.

Appendix: 비용과 토큰도 같은 관측 단위로 본다

외부 API 과금이 없는 로컬 SLM도 모델 실행 시간을 비용으로 환산해 함께 추적

비용 계산 기준

$$\text{compute_seconds} \quad / \quad 3600 \quad \times \quad 0.05 \text{ USD/hour}$$



로컬 SLM 비용

외부 API 과금은 없지만, 모델 실행 시간을 인프라 비용으로 환산해 기록한다



확장 방식

외부 LLM API로 바꾸면 토큰 단가 테이블만 교체하면 그대로 적용된다



핵심 metric

latency · stage duration · tokens · cost · errors 를 한 화면에서 본다

운영자는 비용 증가가 토큰 때문인지, 모델 실행 시간 때문인지, 실패와 연결되는지를 같은 관측 단위에서 함께 확인